

Data Structure Lab9 : Positional Linked List and Array List 2024-2025

Topics

1. Create Position Interface

```
public interface Position<E> {  
    E element(); // Returns the element stored at this position  
}
```

2. Create Positional List interface

```
public interface PositionalList<E> {  
    Position<E> addFirst(E e);  
    Position<E> addLast(E e);  
    Position<E> addBefore(Position<E> p, E e);  
    Position<E> addAfter(Position<E> p, E e);  
    E remove(Position<E> p);  
    Position<E> first();  
    Position<E> last();  
    Position<E> before(Position<E> p);  
    Position<E> after(Position<E> p);  
    int size();  
    boolean isEmpty();  
}
```

3. Create Positional Linked List Using Linked List structure (Nodes)

```
import java.util.LinkedList;
```

```
public class PositionalLinkedList<E> implements PositionalList<E> {  
    private static class Node<E> implements Position<E> {  
        E element;  
        Node<E> prev;  
        Node<E> next;  
  
        public Node(E e, Node<E> p, Node<E> n) {  
            element = e;
```

Data Structure Lab9 : Positional Linked List and Array List 2024-2025

```
        prev = p;
        next = n;
    }

    public E element() {
        return element;
    }
}

private Node<E> header;
private Node<E> trailer;
private int size = 0;

public PositionalLinkedList() {
    header = new Node<>(null, null, null);
    trailer = new Node<>(null, header, null);
    header.next = trailer;
}

public Position<E> addFirst(E e) {
    return addBetween(e, header, header.next);
}

public Position<E> addLast(E e) {
    return addBetween(e, trailer.prev, trailer);
}

public Position<E> addBefore(Position<E> p, E e) {
    Node<E> node = validate(p);
    return addBetween(e, node.prev, node);
}
```

Data Structure Lab9 : Positional Linked List and Array List 2024-2025

```
public Position<E> addAfter(Position<E> p, E e) {  
    Node<E> node = validate(p);  
    return addBetween(e, node, node.next);  
}
```

```
public E remove(Position<E> p) {  
    Node<E> node = validate(p);  
    node.prev.next = node.next;  
    node.next.prev = node.prev;  
    size--;  
    return node.element;  
}
```

```
public Position<E> first() {  
    return header.next == trailer ? null : header.next;  
}
```

```
public Position<E> last() {  
    return trailer.prev == header ? null : trailer.prev;  
}
```

```
public Position<E> before(Position<E> p) {  
    Node<E> node = validate(p);  
    return node.prev == header ? null : node.prev;  
}
```

```
public Position<E> after(Position<E> p) {  
    Node<E> node = validate(p);  
    return node.next == trailer ? null : node.next;  
}
```

```
public int size() {
```

Data Structure Lab9 : Positional Linked List and Array List 2024-2025

```
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    private Position<E> addBetween(E e, Node<E> predecessor, Node<E>
successor) {
        Node<E> newNode = new Node<>(e, predecessor, successor);
        predecessor.next = newNode;
        successor.prev = newNode;
        size++;
        return newNode;
    }

    private Node<E> validate(Position<E> p) {
        if (!(p instanceof Node)) throw new IllegalArgumentException("Invalid
position");
        Node<E> node = (Node<E>) p;
        if (node.prev == null) throw new IllegalArgumentException("Position is
no longer in the list");
        return node;
    }
}
```

4. Implement Basic Methods of Positional Linked List

- addBefore(Position<E> p ,E e)
- addAfter(Position<E> p ,E e)
- remove(Position<E> p)

```
public Iterator<E> iterator() {
```

Data Structure Lab9 : Positional Linked List and Array List 2024-2025

```
return new Iterator<E>() {
    private Position<E> current = first();

    public boolean hasNext() {
        return current != null;
    }

    public E next() {
        if (current == null) throw new NoSuchElementException();
        E element = current.element();
        current = after(current);
        return element;
    }
};
}
```

5. Implement Iterator and Iterable pattern design in Positional Linked Lists

```
import java.util.Arrays;
```

```
public class ArrayList<E> {
    private E[] data;
    private int size = 0;

    public ArrayList() {
        data = (E[]) new Object[10]; // initial capacity
    }

    public void add(E e) {
        if (size == data.length) {
            resize(2 * data.length);
        }
    }
}
```

Data Structure Lab9 : Positional Linked List and Array List 2024-2025

```
        data[size++] = e;
    }

    private void resize(int capacity) {
        E[] newData = (E[]) new Object[capacity];
        System.arraycopy(data, 0, newData, 0, size);
        data = newData;
    }

    public E get(int index) {
        if (index < 0 || index >= size) throw new
IndexOutOfBoundsException();
        return data[index];
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }
}
```

Homework

1. Implement the ArrayList Data structure as it is described in chapter 7.

```
import java.util.Arrays;

public class ArrayList<E> {
    private Object[] elements;
```

Data Structure Lab9 : Positional Linked List and Array List 2024-2025

```
private int size;
private static final int DEFAULT_CAPACITY = 10;

public ArrayList() {
    elements = new Object[DEFAULT_CAPACITY];
    size = 0;
}

public void add(E element) {
    ensureCapacity();
    elements[size++] = element;
}

public E get(int index) {
    checkIndex(index);
    return (E) elements[index];
}

public void remove(int index) {
    checkIndex(index);
    for (int i = index; i < size - 1; i++) {
        elements[i] = elements[i + 1];
    }
    elements[--size] = null; // Clear the last element
}

public int size() {
    return size;
}

private void ensureCapacity() {
    if (size == elements.length) {
```

Data Structure Lab9 : Positional Linked List and Array List 2024-2025

```
        elements = Arrays.copyOf(elements, size * 2);
    }
}

private void checkIndex(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException("Index: " + index + ", Size: " + size);
    }
}
}
```

2. Implement the iterator idea in your ArrayList.

```
import java.util.Iterator;
import java.util.NoSuchElementException;

public class ArrayList<E> implements Iterable<E> {
    // ... (نفس الكود في السؤال ١)

    @Override
    public Iterator<E> iterator() {
        return new ArrayListIterator();
    }

    private class ArrayListIterator implements Iterator<E> {
        private int currentIndex = 0;

        @Override
        public boolean hasNext() {
            return currentIndex < size;
        }
    }
}
```


Data Structure Lab9 : Positional Linked List and Array List 2024-2025

```
@Override
public E next() {
    if (!hasNext()) {
        throw new NoSuchElementException();
    }
    return get(currentIndex++);
}
}
```